

documentacion_obligatorio_2

Ismael Ibarburu y Franco Fernandez

Documentación Técnica - Obligatorio 2

1. Descripción General del Proyecto

TREKAS Delivery es una aplicación web desarrollada con **React, Vite y Vanilla CSS**, diseñada específicamente para operarios y conductores de distribución de galletas y productos de repostería de la empresa TREKAS. La aplicación actúa como una herramienta de logística y pedidos que conecta a los distribuidores con los locales físicos de entrega, permitiendo registrar nuevas órdenes en tiempo real y coordinar su distribución física sobre el terreno.

El sistema se conecta a una API externa para centralizar la información de productos, locales, carritos y pedidos, garantizando la persistencia de los datos en el navegador del usuario a través de tokens de autenticación y sincronizaciones automáticas.

Características Principales:

- **Autenticación de Operarios:** Sistema de Login y Registro seguro con validaciones de cliente.
- **Dashboard con Mapa Interactivo (Leaflet):** Representación visual de locales comerciales. Los marcadores cambian de color según posean pedidos pendientes de entrega ("En camino") o se encuentren al día.
- **Servicio de Geolocalización (GPS):** Acceso al GPS del dispositivo para ubicar al operario y calcular en tiempo real el local más cercano con entregas pendientes.
- **Flujo de Pedidos por Pasos:**
 1. Selección de local de destino ordenado por proximidad.
 2. Catálogo de productos consumido directamente desde la API y la capacidad de agregar o eliminar productos, este carrito se carga a la

base de datos.

3. Confirmación y envío de orden con persistencia de carrito integrada.

- **Historial y Métricas de Conductor:** Registro total de pedidos despachados, filtrado/ordenamiento por precio y fecha, y cambio de perfil de usuario sincronizado con el backend.
- **Manejo de Errores y Rutas Inexistentes:** Interfaz personalizada para páginas 404.

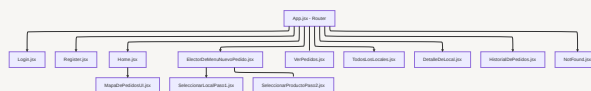
2. Explicación de Pantallas y Componentes Principales

La arquitectura de la aplicación se divide en componentes reutilizables de UI y componentes lógicos organizados bajo una estructura de Single Page Application.

graph TD

```
App[App.jsx - Router] --> Login[Login.jsx]
App --> Register[Register.jsx]
App --> Home[Home.jsx]
App --> NewOrder[ElectorDeMenuNuevoPedido.jsx]
App --> Summary[VerPedidos.jsx]
App --> Locations[TodosLosLocales.jsx]
App --> LocationDetail[DetalleDeLocal.jsx]
App --> Profile[HistorialDePedidos.jsx]
App --> NotFound[NotFound.jsx]

Home --> Mapa[MapaDePedidosUI.jsx]
NewOrder --> Step1[SeleccionarLocalPaso1.jsx]
NewOrder --> Step2[SeleccionarProductoPaso2.jsx]
```



1. Autenticación (Login y Register)

- **Login.jsx:** Formulario limpio de acceso. Utiliza validación nativa y procesamiento de errores de la API. Al acceder con credenciales correctas,

almacena el token y el objeto de datos del usuario en el local storage.

- **Register.jsx:** Formulario con lógica de validación local estricta antes del envío:
 - Verificación de completitud en todos los campos obligatorios.
 - Formato de correo electrónico válido validado mediante expresión regular.
 - Contraseña de seguridad con longitud mínima de 8 caracteres.
 - Verificación de coincidencia de contraseñas.

2. Panel Principal (**Home.jsx**)

Es el centro de control del operario. Cuenta con:

* **MapaDePedidosUI.jsx:** Componente integrado con Leaflet. Mapea geográficamente los locales registrados. Un círculo naranja indica un local con pedidos pendientes de entrega ("EN CAMINO") y un círculo gris indica un local sin actividad. El conductor es representado por un marcador azul. Al presionar sobre un local, se abre un popup que ofrece la opción de acceder al detalle de pedidos de ese local.

* **Localizador de Proximidad:** Hook que obtiene la ubicación actual y calcula la distancia esférica (usa una formula de haversine, que es la manera mas exacta de encontrar el local mas cercano, podriamos haber usado otros metodos de deteccion de cerania en espacios 2D que tienen bajo margen de error pero como la app se podria llegar a expandir en una escala nacional preferimos usar el metodo de distancia esferica) respecto a cada local activo, informando dinámicamente cuál es el local con pedidos pendientes que se encuentra más cerca físicamente (útil para optimizar rutas).

3. Flujo de Nuevo Pedido (**ElectorDeMenuNuevoPedido.jsx**)

Manejado a través de un asistente de dos pasos que mantiene su estado en `sessionStorage` para evitar pérdidas accidentales:

* **Paso 1: Seleccionar Local (**SeleccionarLocalPaso1.jsx**):** Muestra el listado de locales filtrado por un buscador textual y ordenado de menor a mayor distancia del operario (usando la geolocalización). Resalta visualmente el local más cercano porque en teoria seria el local en el que el operario esta agarrando el pedido.

* **Paso 2: Seleccionar Productos (**SeleccionarProductoPaso2.jsx**):** Carga los productos desde la API (`/api/productos`). Cada tarjeta de producto muestra su

imagen (con fallback local si falla la URL), nombre, descripción, precio y un control de cantidad. Este control incluye botones **+** y **-** y un campo de entrada de texto optimizado para dispositivos móviles (`inputMode="numeric"` y `pattern="[0-9]*"`), lo que despliega automáticamente el teclado numérico (numpad) en celulares para facilitar el ingreso manual de cantidades. Los datos se sincronizan en tiempo real con la API del carrito al cambiar el valor o presionar los botones.

4. Resumen de Pedido (`VerPedidos.jsx`)

Presenta el resumen del pedido en curso: local de destino elegido, lista de ítems seleccionados y el precio total. Cuenta con el botón de "Confirmar Pedido", el cual ejecuta la petición POST al servidor, vacía el carrito local/remoto y redirige al Home.

5. Control de Pedidos Activos (`TodosLosLocales.jsx` y `DetalleDeLocal.jsx`)

- **TodosLosLocales.jsx:** Muestra una lista interactiva de locales comerciales ordenados según tengan entregas pendientes en primer lugar. Permite a los operarios marcar un pedido como "Entregado" directamente desde la lista mediante un modal de confirmación emergente.
- **DetalleDeLocal.jsx:** Detalla la información de un local en particular y despliega su historial histórico de envíos, separando y ordenando las entregas pendientes de las ya completadas.

6. Perfil del Operario (`HistorialDePedidos.jsx`)

Muestra el nombre del operario logueado y el conteo de sus órdenes asociadas. Permite:

- * Actualizar o ingresar el nombre del operario en la base de datos a través de una petición PUT a la API.
- * Visualizar el historial de pedidos en formato de tarjetas detalladas, con opciones de ordenamiento por fecha y por tamaño de la compra (precio total).

7. Página 404 (`NotFound.jsx`)

Provee un "catch-all" en el enrutamiento para cualquier URL no registrada, ofreciendo una visualización cuidada del error de navegación y un botón directo de regreso.

3. Endpoints Utilizados de la API

La interacción con el servidor se realiza en formato JSON empleando la cabecera personalizada `x-project-key: trekasa-app` y la cabecera `Authorization: Bearer <token>` cuando la sesión está iniciada.

Método	Endpoint	Componente / Servicio	Propósito
POST	<code>/auth/login</code>	<code>auth.js</code>	Inicia sesión del operario, devuelve el token y datos de perfil.
POST	<code>/auth/register</code>	<code>auth.js</code>	Crea una cuenta de operario y genera el token automático de acceso.
PUT	<code>/api/usuarios/me</code>	<code>auth.js</code>	Modifica los datos de perfil (nombre) del operario logueado.
GET	<code>/api/locales</code>	<code>usarLocales.js</code>	Obtiene el listado de locales comerciales con sus nombres, direcciones y coordenadas geográficas.
GET	<code>/api/productos</code>	<code>SeleccionarProductoPaso2.jsx</code>	Obtiene el listado de catálogo de productos comercializados (Trekas).
GET	<code>/api/carrito</code>	<code>cart.js</code>	Recupera los productos y cantidades guardadas en el

Método	Endpoint	Componente / Servicio	Propósito
			carrito del operario.
PUT	/api/carrito	cart.js	Sincroniza y guarda el estado actual del carrito (productos, cantidades y metadatos) en el servidor.
DELETE	/api/carrito	cart.js	Vacia el contenido completo del carrito del operario al finalizar un pedido.
GET	/api/compras	usarPedidosNuevos.js	Obtiene todos los pedidos registrados en el sistema del proyecto. Usamos COMPRAS para esto porque no queriamos crear un nuevo endpoint, en vez de registrar compras registra registros de pedidos.
PUT	/api/compras/:id	orders.js	Actualiza un pedido en el servidor (usado principalmente para cambiar el estado a "ENTREGADO").

4. Decisiones de Diseño y Desarrollo

Separación de Lógica y UI (Custom Hooks)

Se ha implementado una arquitectura basada en **Custom Hooks**

(e.g. `usarCarrito`, `usarPedidosNuevos`, `usarMapaHome`, `usarFormularioDeRegistro`). Esta decisión separa las responsabilidades de lógica de datos y las de visualización (JSX), permitiendo hacer pruebas sobre la lógica del negocio de manera aislada y simplificando el mantenimiento de las pantallas de UI. Manteniendo la legibilidad del documento.

Persistencia Híbrida del Estado

- **localStorage:** Utilizado para almacenar el token de la sesión del usuario y sus datos básicos de identificación, garantizando que el usuario mantenga su sesión iniciada al cerrar y abrir el navegador.
- **sessionStorage:** Utilizado para recordar el local de destino seleccionado (`localSeleccionado`) durante el flujo de creación de pedido, permitiendo la navegación fluida entre los pasos sin perder la selección, pero borrándolo limpiamente al confirmar la orden o cerrar la pestaña.
- **Persistencia Remota (API):** El carrito de compras se persiste directamente en el servidor. Esto permite cambiar de dispositivo o recargar la página sin perder la selección de ítems del pedido.

Integración de Mapas y GPS Offline (Fórmula de Haversine)

Para resolver la proximidad de entrega sin depender de APIs de pago externas (como Google Maps Distance Matrix), se utiliza la

Fórmula de Haversine

implementada directamente en JavaScript para calcular la distancia en kilómetros sobre la esfera terrestre entre las coordenadas provistas por la geolocalización HTML5 del navegador y las coordenadas fijas del local:

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \text{lat}}{2} \right) + \cos(\text{lat}_1) \cos(\text{lat}_2) \sin^2 \left(\frac{\Delta \text{lon}}{2} \right)} \right)$$

Esto permite ordenar los locales al instante de forma local y eficiente.

Es un metodo complejo computacionalmente pero permite escalar la app en un futuro, hay metodos mas eficientes que usan las coordenadas en un plano 2D

pero si queremos expandir la app en escala nacional es util para medir con eficacia, aparte, el numero de locales nunca va a ser demasiado grande como para que su peso computacional se notable.

5. Instrucciones para Ejecutar el Proyecto

Requisitos Previos

- Tener instalado **Node.js** (versión 18 o superior recomendada).
- Gestor de paquetes **npm** (incluido por defecto con Node.js).

Instalación y Ejecución

1. Abrir una terminal en la carpeta `/codigo-app` (donde se encuentra el archivo `package.json`).
2. Instalar las dependencias del proyecto ejecutando:

```
bash npm install
```
3. Iniciar el servidor de desarrollo local con:

```
bash npm run dev
```
4. Abrir en el navegador la dirección indicada en la consola (normalmente `http://localhost:5173`).

Construcción para Producción

Para empaquetar y optimizar la aplicación para su distribución y subida a un servidor (ej. Vercel, Netlify):

```
npm run build
```

Los archivos compilados listos para servir se guardarán en la carpeta `/dist`.

7. Declaración del Uso de Inteligencia Artificial (IAG)

En cumplimiento con las directivas del curso sobre el uso ético y transparente de herramientas de Inteligencia Artificial Generativa:

- **Herramienta Utilizada:** Antigravity (asistente de programación basado en modelos de Google DeepMind).
- **Uso Detallado:**

- **Análisis inicial:** Lectura de las pautas en PDF del obligatorio 2 y contraste contra la base de código para identificar los faltantes funcionales y de estructura.
- **Desarrollo de componentes:** La IA fue fundamental en el proceso de debugging y creación de la arquitectura base de la aplicación. Fue utilizada para organizar el trabajo y para resolver dificultades de comprensión del código. Fue un trabajo conjunto entre la IA que a veces no hace las mejores prácticas y el trabajo humano para asegurarnos de tener un código legible y limpio para ser usado por humanos.
- **Documentación:** Estructuración y redacción inicial de esta memoria técnica.